TicketDesk — Solution

Category: WEB | Points: 700 | Author: Jerit3787

Overview

A blind, second-order stored XSS fired by a headless-Chromium admin bot, under a strict CSP with an HttpOnly admin cookie and no network egress — so the exploit must run same-origin and exfiltrate in-band. The reporter stores a `<script src>` in a ticket field that only the admin view renders unescaped, serves the script from the app's own origin via the file-upload gadget, and has the admin's browser read the admin-only flag and post it back as a comment on the reporter's ticket.

The flag endpoint is not a one-shot read: `/admin/flag` requires a per-session `X-Ops-Token` that is only rendered on the admin-only `/admin/console` page. So the payload must run **two** admin-context requests — fetch the console, scrape the token, then request the flag with it — and the reporter learns this requirement blind, because their first attempt posts the endpoint's error back as a comment on their own ticket.

Steps

1. Log in as `reporter` / `report2026` (each login gets its own private ticket space). No source is shipped; everything below is found by probing.
2. Notice three walls: the CSP response header is `script-src 'self'` (no inline), the session cookie is HttpOnly (no cookie theft), and nothing external is reachable (no external exfil).
3. Find the **blind second-order sink** by trying HTML in every field: `environment` is escaped in your own ticket view but is rendered raw in the admin view the bot opens.
4. Find the **CSP-bypass gadget**: the ticket page offers a screenshot upload restricted to images in the UI only; the server accepts any file, and `/uploads/<name>.js` is served as `text/javascript` from the same origin, which `script-src 'self'` allows.
5. Upload `payload.js` with a direct multipart POST (bypassing the image-only UI).
6. File a ticket with `environment = <script src="/uploads/<name>.js"></script>`.
7. **First attempt** with `fetch('/admin/flag')` -> the admin bot runs it and posts back `{"error": "missing or invalid X-Ops-Token (issued per session at /admin/console)"}` as a comment on your ticket. This is the only way you learn the second obstacle — you never see any admin response directly.
8. **Second stage:** the payload `fetch('/admin/console')` (admin-only, so only the bot can read it), scrapes the `ops-token` meta tag, then requests `/admin/flag` with the `X-Ops-Token` header and POSTs the flag to `/ticket/<id>/comment`.
9. Reload your own ticket -> the admin's comment contains the flag.

See `payload.js` and `solve.py` for the automated chain.

Red Herrings

- **Inline XSS / `onerror`** — blocked by CSP `script-src 'self'`.
- **Cookie theft (`document.cookie`)** — admin cookie is HttpOnly.
- **Exfil to an external server** — no egress; must exfil in-band.
- **Testing on your own ticket** — `environment` is escaped in the reporter view; the payload only fires in the admin view you cannot see.
- **Image-only upload UI** — a client-side restriction; the server enforces nothing.
- **Other players' tickets** — isolated per login; you cannot read anyone else's flag comment.

- **Guessing / reusing the ops token** — it is random per admin session and checked with a constant-time compare; a reporter cannot reach `/admin/console` (403) to read it, so the token can only be obtained by running code in the admin's context.
- **One-shot `fetch('/admin/flag')`** — returns a 403 error, not the flag; the console round-trip is mandatory.

Flag

```
gctf26{h4v3_y0u_tr13d_turn1ng_th3_4dm1n_0ff_4nd_0n}
```